

⇒ Data Structures ⇒

Definition

A data structure is a way of organizing and storing data so that it can be used efficiently. It helps in optimizing the performance of programs by allowing efficient access, modification, and processing of data.

All Data Structures Overview

Data Structure	Diagram / Representation	Description
1. Array		Stores elements of the same type in contiguous memory locations.
2. Linked List		Stores elements in nodes. Each node points to the next node.
3. Stack (LIFO)		Follows LIFO (Last In, First Out) principle. Insertion & deletion at the top.
4. Queue (FIFO)		Follows FIFO (First In, First Out) principle. Insertion at rear, deletion at front.
5. Tree		Hierarchical structure with a root node and zero or more subtrees.
6. Graph		Collection of nodes (vertices) connected by edges.
7. Hash Table (Hash Map)		Stores key-value pairs using a hash function for fast access.
8. Heap		A special tree-based structure (Min-Heap or Max-Heap).

Pro Tip: Understanding the right data structure is the key to writing efficient and optimized code. ☆

≧ What is an Algorithm? ≦

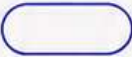
Definition

An algorithm is a finite set of well-defined instructions or steps to solve a problem or accomplish a task. It must produce output for given input and terminate after a finite number of steps.

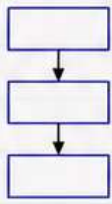
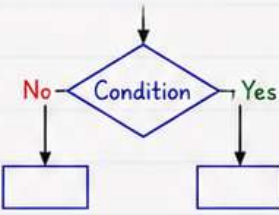
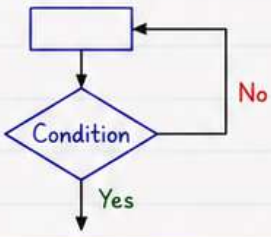
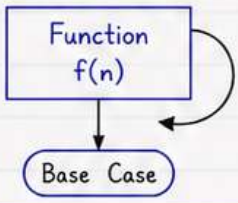
Characteristics of an Algorithm

1. Input : Takes zero or more inputs.
2. Output : Produces at least one output.
3. Definiteness : Each step is clear and unambiguous.
4. Finiteness : Terminates after a finite number of steps.
5. Effectiveness : Each step is basic and executable in finite time.

Basic Symbols in Algorithms / Pseudocode

<u>Symbol</u>	<u>Name</u>	<u>Use</u>
	Terminator	Indicates start or end of the algorithm.
	Input / Output	Represents input or output.
	Process	Represents an instruction or step.
	Decision	Represents a decision point with Yes/No (True/False) paths.
	Flow Line	Shows the direction of flow.
	Connector (On-page)	Connects flow lines on the same page.
	Connector (Off-page)	Connects flow lines to another page.

Types of Algorithms

1. Sequence Steps are executed one after another. 	2. Selection (Decision) Choice is made based on a condition. 	3. Iteration (Loop) Steps are repeated until a condition is met. 	4. Recursion A function calls itself to solve smaller instances of the same problem. 
---	--	---	--

Pro Tip:

A good algorithm is correct, efficient, simple to understand, and solves the problem with optimal time and space complexity.



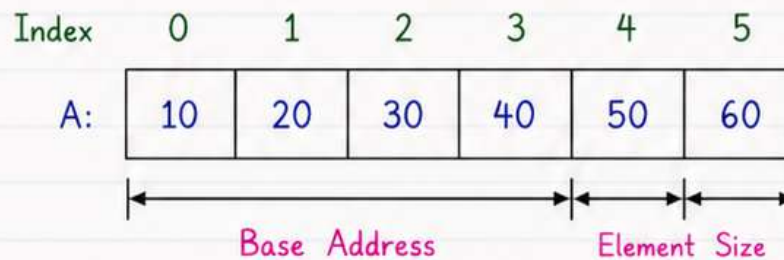
Arrays

Definition

An array is a collection of elements of the same type stored at contiguous memory locations. Each element can be accessed using an index.

Representation

Array: A



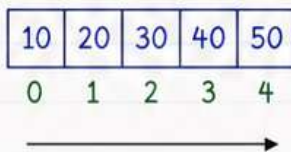
Characteristics

1. Fixed Size : Size is fixed once declared.
2. Homogeneous : All elements are of the same type.
3. Contiguous Memory : Elements are stored in adjacent memory locations.
4. Random Access : Any element can be accessed directly using its index.
5. Efficient Access : Access time is $O(1)$ for any element.

Basic Operations

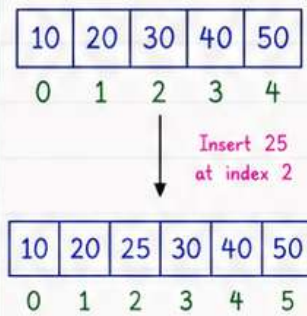
1. Traversal

Visit each element of the array.



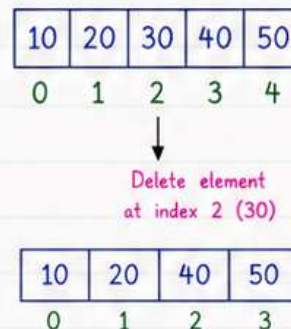
2. Insertion

Add an element at a specified position.



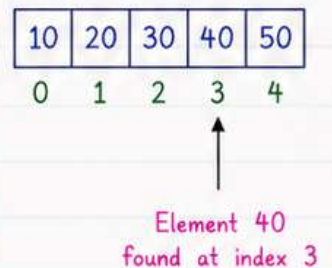
3. Deletion

Remove an element from a position.



4. Searching

Find the index of a given element.



Applications

- Storing lists of data such as marks, scores, names, etc.
- Implementing other data structures like stacks, queues, heaps, etc.
- Used in searching, sorting, matrix operations, lookup tables, etc.

Pro Tip:

Arrays provide fast access to elements by index, but inserting or deleting in the middle can be costly as elements need to be shifted.

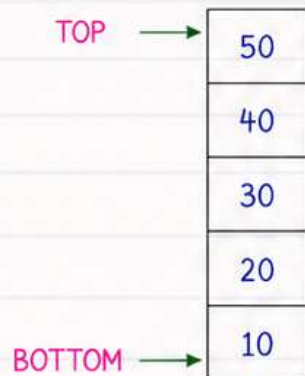


≧ Stacks ≦

Definition

A stack is a linear data structure that follows the **LIFO** (Last In, First Out) principle. Elements are added and removed from the same end, called the top.

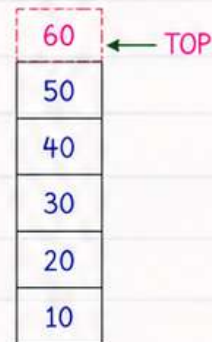
Representation



- Top points to the topmost element.
- New elements are added or removed from the top.

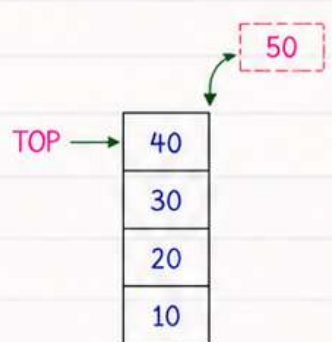
Push Operation

Add element 60



Pop Operation

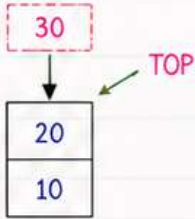
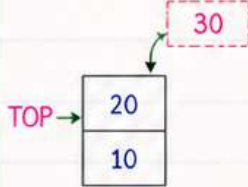
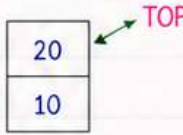
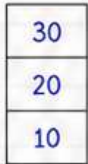
Remove element 50



Characteristics

1. Follows LIFO (Last In, First Out) principle.
2. Insertion and deletion occur only at the top.
3. **Dynamic size** : Can grow or shrink during program execution.
4. **Efficient operations** : Push and pop are $O(1)$.
5. Used in function calls, expression evaluation, backtracking, etc.

Basic Operations

1. Push Add an element to the top. 	2. Pop Remove the top element. 	3. Peek / Top Get the value of top element.  Return : 20	4. isEmpty Check whether stack is empty.  Return : True	5. isFull Check whether stack is full.  Return : True
---	---	---	--	--

Applications

- Function call management (Call Stack).
- Expression evaluation and conversion (infix $\rightleftharpoons$ postfix/prefix).
- Parentheses matching and syntax parsing.
- Backtracking (e.g., maze, N-Queens problem).
- Undo operations in editors.
- Browser history navigation.

Pro Tip:

Always make sure to check for overflow before push and underflow before pop to avoid errors!



Queues

Definition

A queue is a linear data structure that follows the FIFO (First In, First Out) principle. Elements are added at the rear and removed from the front.

Representation



Front indicates the first element (to be removed).

Rear indicates the last element (where new element is added).

Characteristics

1. Follows FIFO (First In, First Out) principle.
2. Insertion occurs only at the rear.
3. Deletion occurs only at the front.
4. Dynamic size : Can grow or shrink during execution.
5. Efficient operations : Enqueue and Dequeue are $O(1)$.

Basic Operations

1. Enqueue	2. Dequeue	3. Front/Ppeek	4. isEmpty	5. isFull
Add an element at the rear.	Remove the element from the front.	Get the value of front element.	Check whether the queue is empty.	Check whether the queue is full (for fixed-size).
		Return : 10	Return : True	Return : True

Applications

- CPU scheduling and task management.
- Handling requests in web servers.
- Printer queue (print spooling).
- Breadth-First Search (BFS) in graphs.
- Simulation of waiting lines (e.g., ticket booking, call centers).

≡ Strings ≡

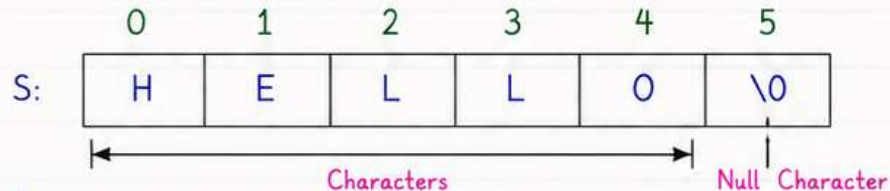
Definition

A string is a sequence of characters. In DSA, a string is an array of characters terminated by a null character '\0'.

Representation

String: "HELLO"

Index



Characteristics

1. **Sequence of Characters** : An ordered collection of characters.
2. **Null Terminated** : In C/C++, a string ends with a null character '\0'.
3. **Immutable in Many Languages** : Strings are immutable in Java, Python, etc.
4. **Variable Length** : Length can vary from 0 to n characters.
5. **Case Sensitive** : 'A' and 'a' are different in strings.
6. **Stored in Contiguous Memory** : Characters are stored in adjacent memory locations.

Basic Operations

1. Traversal Visit each character of the string. String: "HELLO" <table><tr><td>H</td><td>E</td><td>L</td><td>L</td><td>O</td><td>\0</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table> →	H	E	L	L	O	\0	0	1	2	3	4	5	2. Length Find the number of characters. String: "HELLO" <table><tr><td>H</td><td>E</td><td>L</td><td>L</td><td>O</td><td>\0</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table> Length = 5	H	E	L	L	O	\0	0	1	2	3	4	5	3. Concatenation Join two strings. S1: "HELLO" S2: "WORLD" Result: "HELLOWORLD"	4. Comparison Compare two strings. S1: "HELLO" S2: "HELLO" Result: Equal	5. Substring Extract a part of the string. String: "HELLO" Substring (1 to 3): "ELL"
H	E	L	L	O	\0																							
0	1	2	3	4	5																							
H	E	L	L	O	\0																							
0	1	2	3	4	5																							

Common Methods (C++ Style)

Method	Description	Method	Description
s.length()	Returns the length of the string	s.compare(str)	Compares two strings
s.empty()	Checks if the string is empty	s.find(ch/str)	Returns index of first occurrence
s.clear()	Removes all characters	s.insert(pos, str)	Inserts str at position pos
s.append(str)	Appends str at the end	s.erase(pos, len)	Erases len characters from pos
s.substr(pos, len)	Returns substring from pos of length len	s.replace(pos, len, str)	Replaces len characters with str

Applications

- Text processing, searching, and pattern matching.
- Data validation (emails, phone numbers, passwords, etc.).
- Compression, encryption, and encoding.
- Used in editors, compilers, databases, and many real-world applications.

Example:

String s = "DATA";
Length = 4

Pro Tip:

Understand the difference between character and string.

A character is a single entity, while a string is a collection of characters.

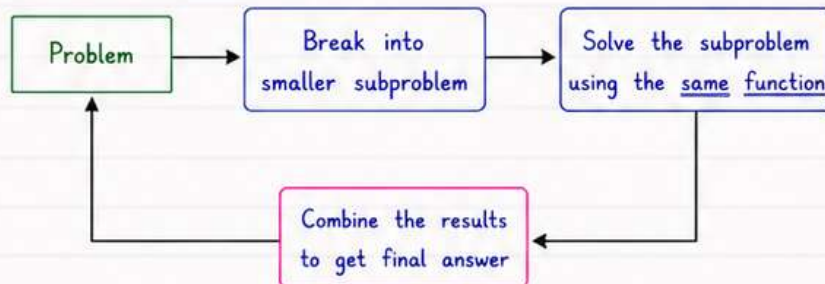


≡ Recursion ≡

Definition

Recursion is a programming technique where a function calls itself to solve a problem by breaking it down into smaller subproblems of the same type.

How Recursion Works



Key Points

- **Base Case** : The condition under which recursion stops.
- **Recursive Case** : The condition where the function calls itself with a smaller input.
- Every recursion must have a base case, otherwise it leads to infinite calls (Stack Overflow).

Example 1: Factorial of n ($n!$)

```
Factorial of n:
if n == 0 or n == 1:
    return 1 ← Base Case
else:
    return n * factorial(n-1)
    ← Recursive Case
```

Dry Run ($n = 4$)

```
fact(4) = 4 * fact(3)
        = 4 * (3 * fact(2))
        = 4 * (3 * (2 * fact(1)))
        = 4 * (3 * (2 * 1))
        = 4 * 3 * 2 * 1
        = 24
```

Recursion Call Stack

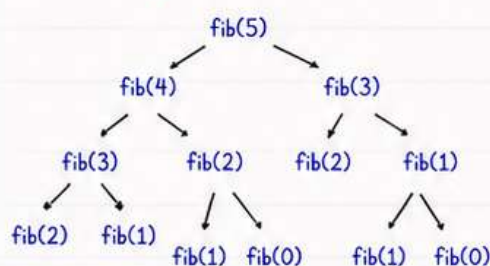
Top →	fact(1)	= 1
	fact(2)	= 1 * 2
	fact(3)	= 2 * 3
	fact(4)	= 6 * 4

Calls are added on top and removed when returned.

Example 2: Fibonacci Series

```
Fibonacci (nth term):
if n == 0:
    return 0 ← Base Case
if n == 1:
    return 1 ← Base Case
else:
    return fib(n-1) + fib(n-2)
    ← Recursive Case
```

Dry Run ($n = 5$)



Output:

Fibonacci series: 0, 1, 1, 2, 3, 5, ...

fib(5) = 5

Characteristics / Points to Remember

1. Recursion is a top-down approach.
2. It uses more memory (call stack).
3. Can be solved using iteration (bottom-up) too.
4. Useful for problems with overlapping subproblems.
5. Make sure to write correct base case.
6. Excessive recursion may cause stack overflow.

Applications

- Tree / Graph traversals
- Divide and conquer algorithms
- Backtracking (N-Queens, Sudoku Solver)
- Dynamic Programming (with memoization)
- Problems that can be broken into smaller subproblems

Pro Tip:

Always identify the base case first, then think how to reduce the problem size and call the same function.

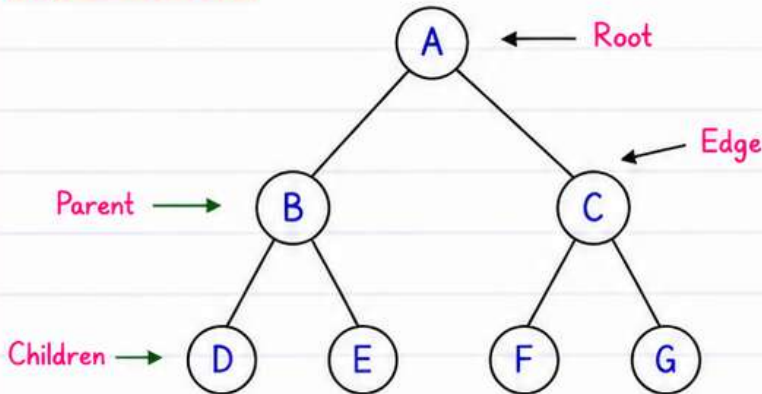


Trees

Definition

A tree is a non-linear data structure consisting of nodes connected by edges. It has a hierarchical structure with a single root node and zero or more subtrees.

Representation



- **Root** : Topmost node in the tree.
- **Edge** : Connection between two nodes.
- **Parent** : A node that has one or more children.
- **Child** : A node that is connected to a parent node.
- **Leaf Node** : A node with no children.
- **Subtree** : A tree formed by a node and all its descendants.

Characteristics

1. A tree has exactly one root node.
2. Each node (except the root) has exactly one parent.
3. A node can have zero or more children.
4. There are no cycles in a tree.
5. A tree with n nodes has exactly $n - 1$ edges.

Basic Operations

1. Traversal Visit each node in a specific order. Types : • Preorder • Inorder • Postorder • Level Order	2. Insertion Add a new node to the tree. Insert F	3. Deletion Remove a node from the tree. Delete E	4. Searching Find a node with a given value. Search C	5. Finding Parent / Child Identify parent or children of a node. Parent of B is A Children of B : D, E
--	--	--	--	--

Applications

- Representing hierarchical data (e.g., file systems, organization charts).
- Expression trees in compilers.
- Database indexing (e.g., B-trees).
- Decision making (e.g., game trees, AI).

Graphs

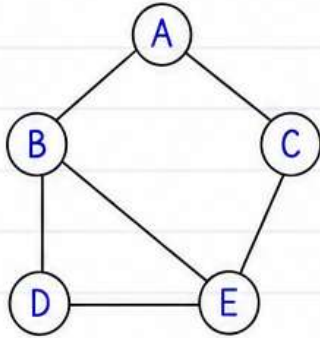
Definition

A graph is a non-linear data structure consisting of vertices (nodes) and edges. It can be directed or undirected.

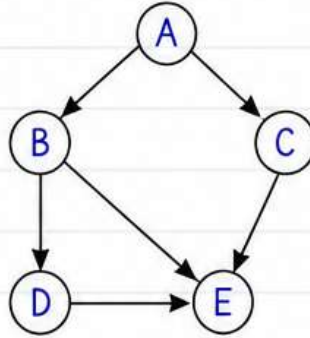
Graphs can be used to represent networks and relationships.

Representation

Undirected Graph



Directed Graph



- **Vertex (Node)** : An individual point in the graph.
- **Edge** : A connection between two vertices.
- **Directed Graph** : Edges have a direction.
- **Undirected Graph** : Edges have no direction.
- **Weighted Graph** : Edges have associated weights or costs.
- **Connected Graph** : There is a path between every pair of vertices.

Characteristics

1. A graph consists of a set of vertices (V) and a set of edges (E).
2. Edges can be directed (one direction) or undirected (no direction).
3. A graph can be weighted or unweighted.
4. A graph can be connected or disconnected.
5. A simple graph has no self-loops and no parallel edges.
6. Used to model real-world networks and relationships.

Basic Operations

1. Add Vertex	2. Add Edge	3. Remove Vertex	4. Remove Edge	5. Check Adjacency
Add a new vertex to the graph.	Add an edge between two vertices.	Delete a vertex and its incident edges.	Delete an edge between two vertices.	Determine if two vertices are adjacent.

Applications

- Social networks (friends, followers, connections).
- Road maps and navigation systems.
- Web link analysis and ranking (e.g., PageRank).
- Network design and communication systems.
- Recommendation systems.
- Dependency resolution in build systems.

DSA SERIES SUMMARY

We've covered all the important Data Structures & key concepts that form the foundation of DSA.

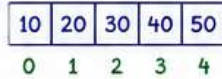
Keep Practicing. Keep Building. Keep Growing! 🚀

1. ALGORITHM



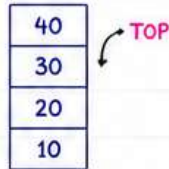
- Step-by-step procedure to solve a problem.
- **Properties:** Input, Output, Definiteness, Finiteness, Effectiveness.

2. ARRAY



- Collection of same type of elements.
- Stored in contiguous memory.
- Fast access using index.

3. STACK



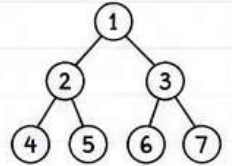
- LIFO (Last In First Out)
- **Operations:** Push, Pop, Peek

4. QUEUE



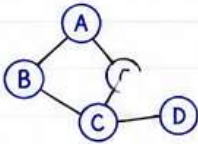
- FIFO (First In First Out)
- **Operations:** Enqueue, Dequeue

5. TREE



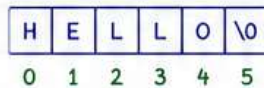
- Hierarchical structure.
- Root, Node, Leaf, Edge, Subtree.

6. GRAPH



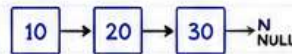
- Collection of vertices (nodes) and edges.
- Can be directed / undirected, weighted / unweighted.

7. STRING



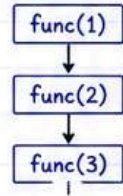
- Sequence of characters terminated by '\0'.
- **Operations:** Traversal, Length, Concatenation, Comparison, Substring.

8. LINKED LIST



- Collection of nodes (data + address).
- Nodes are not stored in contiguous memory.
- **Types:** Singly, Doubly, Circular.

9. RECURSION



- Function calling itself to solve smaller subproblems.
- Needs **Base Case**.

KEY TAKEAWAYS

- ✓ Understand the basics clearly.
- ✓ Practice problems daily.
- ✓ Visualize before you code.
- ✓ Break the problem into smaller parts.
- ✓ Analyze time & space complexity.

- ✓ Solve easy → medium → hard.
- ✓ Revise & implement multiple times.
- ✓ Learn from mistakes.
- ✓ Consistency beats talent.
- ✓ Build projects and apply your knowledge.

WHERE TO USE DSA ?

- ★ Problem Solving
- ★ Coding Interviews
- ★ Competitive Programming
- ★ Building Efficient Applications
- ★ Optimizing Code
- ★ System Design
- ★ Real-world Projects

MOTIVATION



DSA is not about learning, it's about practicing. The more you practice, the better you become! 💪

💖 If you found this helpful, 💖

LIKE | **SHARE** | **COMMENT**

— and —

FOLLOW @aman_views

for more DSA notes & future interviews! 🚀



SAVE THIS POST FOR FUTURE REFERENCE

